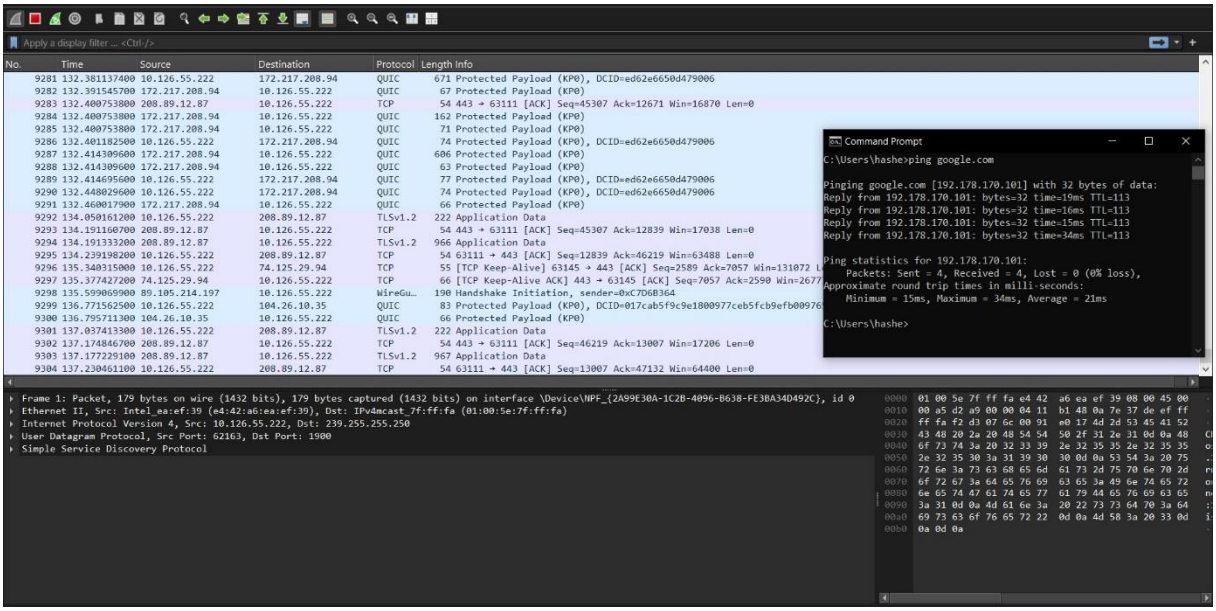
Network Protocol Analysis and Performance Diagnostic Report

Introduction

This report documents a technical investigation into the network infrastructure of the University of the People Uo People following reports of persistent network latency. As a junior network engineer, I utilized Wireshark to capture and analyze real-time traffic, aiming to identify the protocols in operation and determine their impact on performance. According to Sharpe et al. (n.d.), gaining visibility into data flows is essential for diagnosing bottlenecks. By capturing and filtering real network traffic, I was able to observe exactly how data flows across the organization's network to identify potential issues.

Methodology

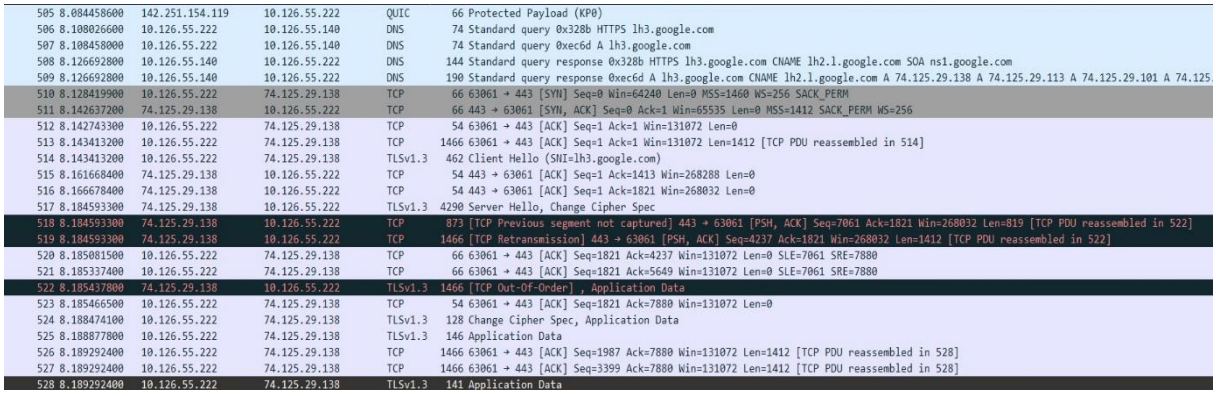
The diagnostic process involved capturing live packets on the WLAN interface while interacting with the Uo People student portal. To verify external connectivity, I executed a Ping test to google.com. Conceptually, a ping test works like knocking on a friend's door and calling his name; if he answers, you know he is "home" and active. In network terms, the successful reply from Google confirmed that the connection was functional, showing 0% packet loss (ChemiCloud, 2020). After the capture, I applied specific filters to study the protocols in detail.



Protocol Identification

During the session, I identified several protocols essential for communication and security. Below are five specific protocols observed:

- **WireGuard (VPN Protocol):** Observed in my capture. WireGuard is a modern protocol for secure data encapsulation. The Handshake Initiation represents the initial handshake required to open a secure tunnel (Angelo, 2019).
- **TLSv1.3 (Transport Layer Security):** This acts like a sealed envelope for data, ensuring that the application data remains encrypted and private. TLS 1.3 is the latest version, offering faster handshakes.
- **DNS (Domain Name System):** DNS works like a phonebook for the internet, translating human-readable names like google.com into IP addresses that devices can understand.
- **QUIC (Quick UDP Internet Connections):** QUIC is designed for speed, reducing latency by streamlining the connection process between the client and the server.
- **TCP (Transmission Control Protocol):** The backbone of reliable data transfer, ensuring that all web portal elements are delivered without errors.

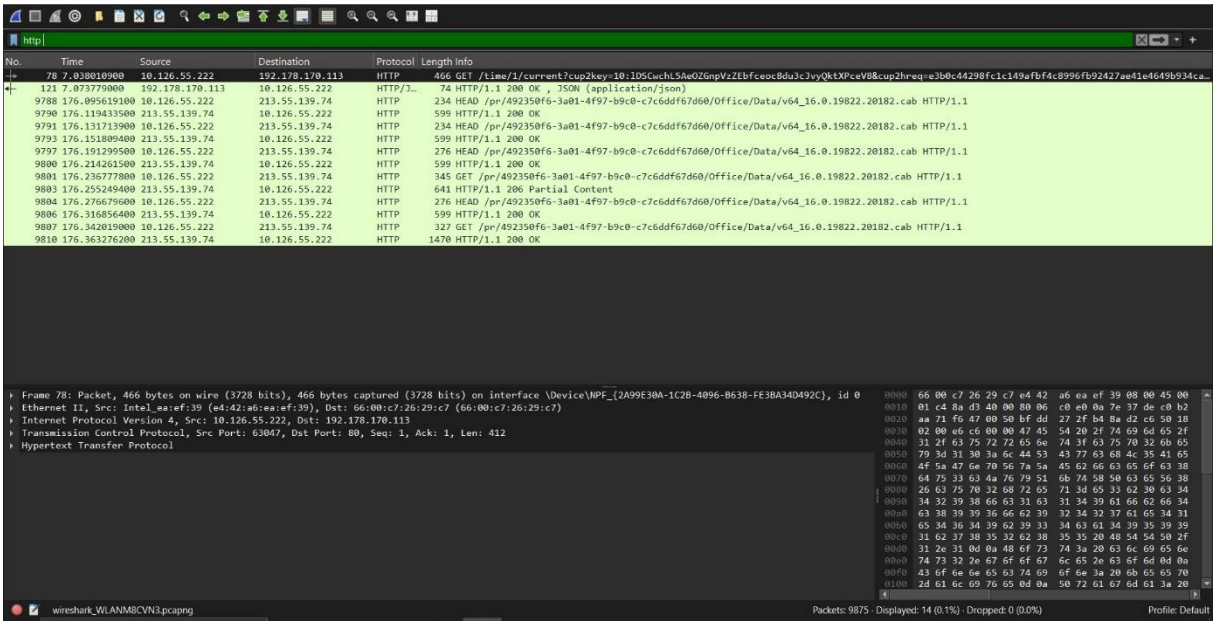


13	2.990498000	10.126.55.222	50.85.242.216	TCP	1478 63046 → 443 [ACK] Seq=378 Ack=8582 Win=132352 Len=1424 [TCP PDU reassembled in 14]
14	2.990498000	10.126.55.222	50.85.242.216	TLSv1.2	508 Application Data
15	2.991128600	10.126.55.222	50.85.242.216	TCP	1478 63046 → 443 [ACK] Seq=2256 Ack=8582 Win=132352 Len=1424 [TCP PDU reassembled in 16]
16	2.991128600	10.126.55.222	50.85.242.216	TLSv1.2	449 Application Data
17	3.030178000	50.85.242.216	10.126.55.222	TCP	54 443 → 63046 [ACK] Seq=8582 Ack=2256 Win=4195072 Len=0
18	3.032118400	50.85.242.216	10.126.55.222	TCP	54 443 → 63046 [ACK] Seq=8582 Ack=4075 Win=4195072 Len=0
19	3.033101400	50.85.242.216	10.126.55.222	TLSv1.2	87 Encrypted Handshake Message
20	3.033385700	10.126.55.222	50.85.242.216	TLSv1.2	277 Encrypted Handshake Message
21	3.072963600	50.85.242.216	10.126.55.222	TCP	5750 443 → 63046 [ACK] Seq=8615 Ack=4298 Win=4194816 Len=5696 [TCP PDU reassembled in 24]
22	3.072963600	50.85.242.216	10.126.55.222	TCP	124 [TCP Previous segment not captured] 443 → 63046 [PSH, ACK] Seq=17159 Ack=4298 Win=4194816 Len=70 [TCP PDU reassembled in 24]
23	3.073141800	10.126.55.222	50.85.242.216	TCP	66 63046 → 443 [ACK] Seq=4298 Ack=14311 Win=132352 Len=0 SLE=17159 SRE=17229
24	3.074435300	50.85.242.216	10.126.55.222	TLSv1.2	2902 [TCP Out-Of-Order] , Encrypted Handshake Message
25	3.074613100	10.126.55.222	50.85.242.216	TCP	54 63046 → 443 [ACK] Seq=4298 Ack=17229 Win=132352 Len=0
26	3.080641500	10.126.55.222	50.85.242.216	TLSv1.2	267 Encrypted Handshake Message, Change Cipher Spec, Encrypted Handshake Message
27	3.112208600	50.85.242.216	10.126.55.222	TLSv1.2	129 Change Cipher Spec, Encrypted Handshake Message
28	3.114199800	50.85.242.216	10.126.55.222	TCP	1019 [TCP Previous segment not captured] 443 → 63046 [PSH, ACK] Seq=18728 Ack=4511 Win=4194560 Len=965
29	3.114254400	10.126.55.222	50.85.242.216	TCP	66 63046 → 443 [ACK] Seq=4511 Ack=17304 Win=132352 Len=0 SLE=18728 SRE=19693
30	3.118627500	50.85.242.216	10.126.55.222	TCP	1478 [TCP Out-Of-Order] 443 → 63046 [ACK] Seq=17304 Ack=4511 Win=4194560 Len=1424
31	3.118718200	10.126.55.222	50.85.242.216	TCP	54 63046 → 443 [ACK] Seq=4511 Ack=19693 Win=132352 Len=0
32	3.137012800	10.126.55.222	50.85.242.216	TCP	1478 63046 → 443 [ACK] Seq=4511 Ack=19693 Win=132352 Len=1424 [TCP PDU reassembled in 33]
33	3.137012800	10.126.55.222	50.85.242.216	TLSv1.2	495 Application Data
34	3.172510100	50.85.242.216	10.126.55.222	TCP	54 443 → 63046 [ACK] Seq=19693 Ack=6376 Win=4195072 Len=0
35	3.335802600	50.85.242.216	10.126.55.222	TLSv1.2	563 Application Data

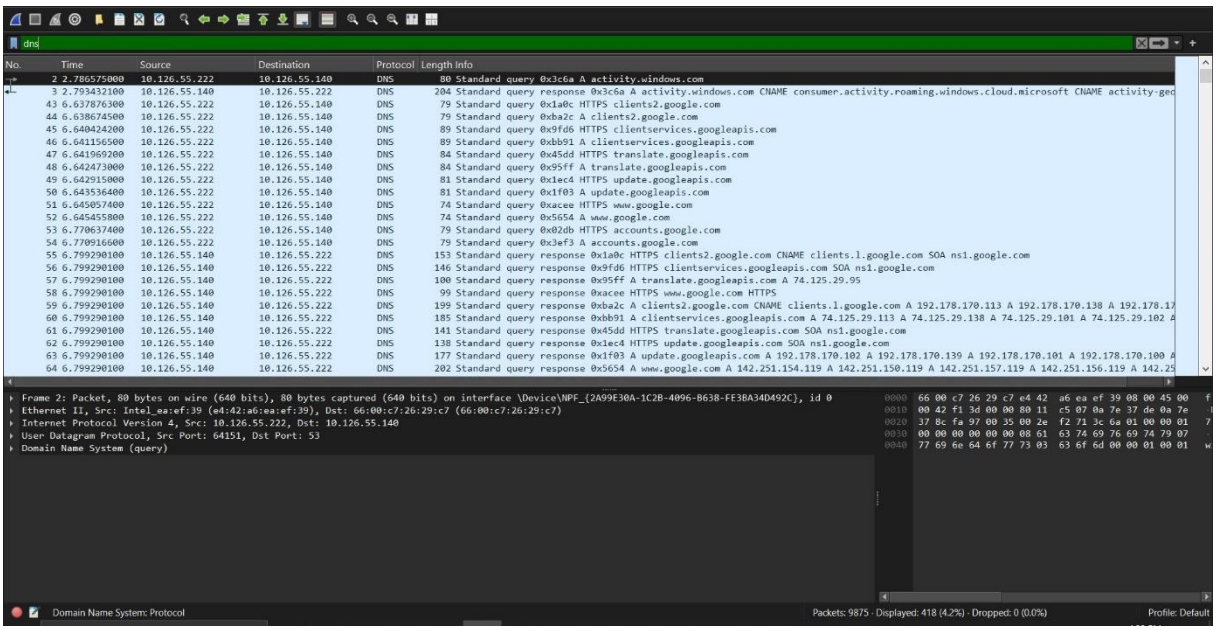
Quantitative Packet Analysis

To investigate the slow network complaint, I quantified the traffic volume for specific protocols using Wireshark's display filters:

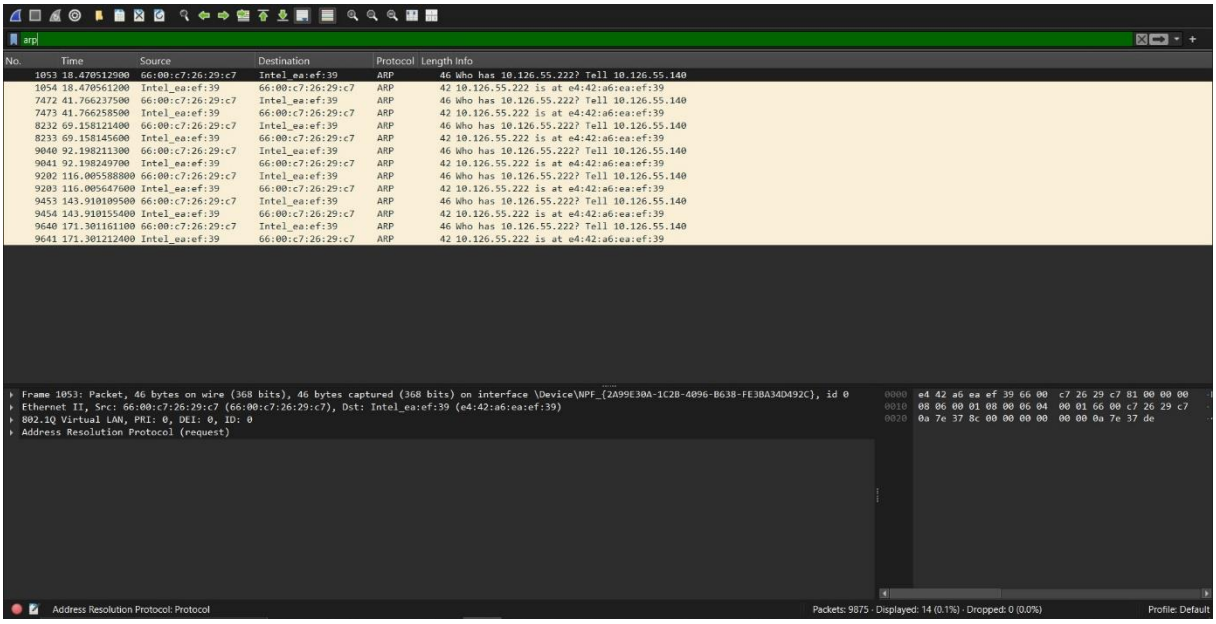
HTTP: Filtering for http revealed 14 packets. These represent the actual web page requests and responses between my device and the server.



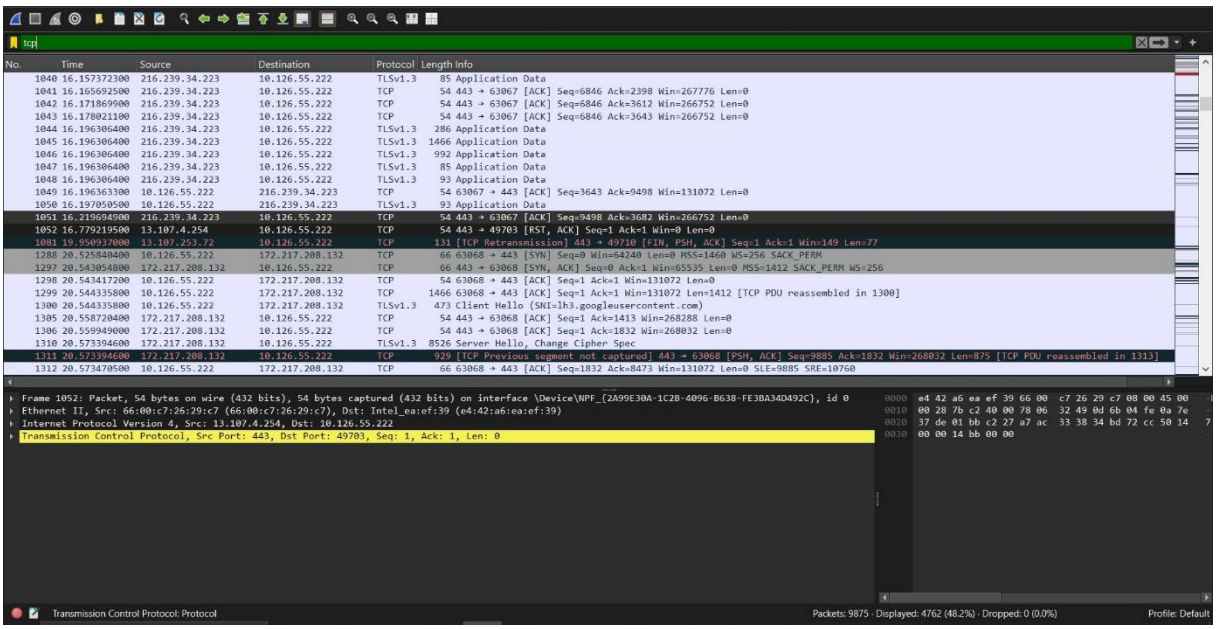
DNS: Filtering for dns showed 418 packets. Since every website visit requires a DNS query first, a high volume here is expected, though it can cause perceived slowness if the phonebook lookup is delayed



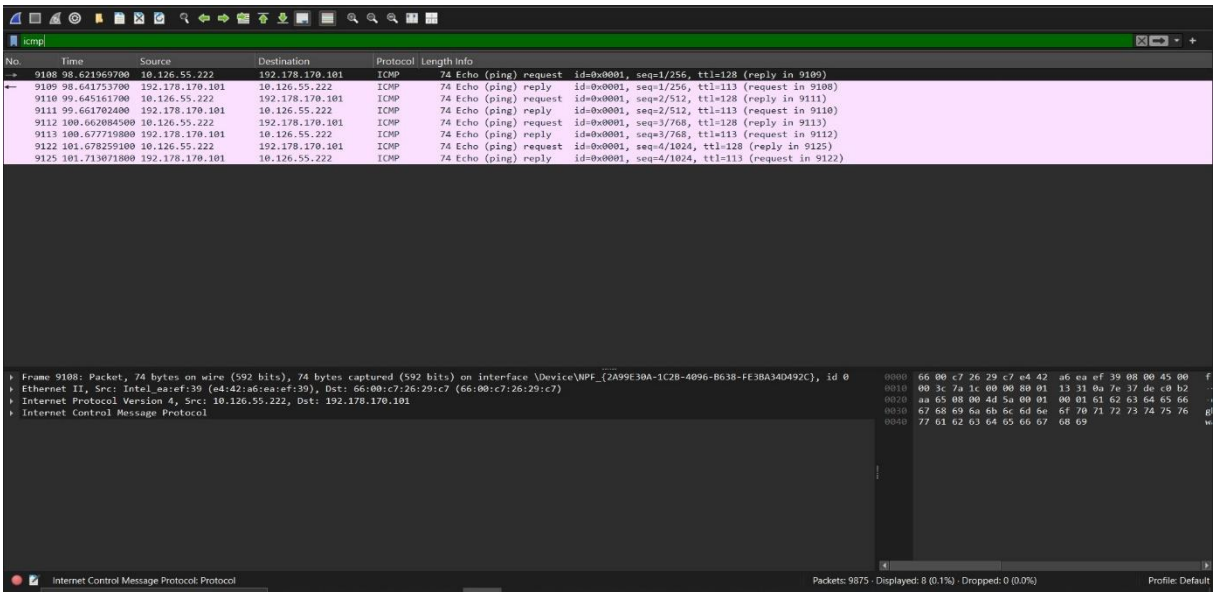
ARP: I observed 14 ARP packets. ARP is responsible for mapping IP addresses to physical MAC addresses, helping devices on the local network identify and communicate with each other



TCP: Filtering for tcp showed 4,762 packets, accounting for 48.2% of the total capture. TCP requires an acknowledgment for every packet sent, which naturally increases the packet count but ensures reliability.



ICMP: I captured 8 packets directly from the ping test, confirming stable connectivity with 4 requests and 4 replies.



Conclusion

The analysis shows a healthy variety of protocols. The high volume of TCP packets suggests that most of the network activity was dedicated to reliable data transmission. However, the "slow network" complaint may be related to how traffic is prioritized. As Swyx (2020b) suggests, effective Traffic Engineering is required to manage these flows. Furthermore, the use of VPNs (WireGuard), while providing security, adds an extra layer of encapsulation that can impact throughput (Jadhav & Sheth, 2021). Tools like Wireshark are essential for any network engineer to understand what is happening inside a network and resolve performance issues.

References

Angelo, R. (2019). Secure protocols and virtual private networks: An evaluation. *Issues in Information Systems, 20*(3), 37-46.

ChemiCloud. (2020, August 9). *How to perform a ping test in windows, mac OS, and linux*. https://chemicloud.com/kb/article/how-to-perform-a-ping-test/

Jadhav, R. R., & Sheth, P. S. (2021). VPN: Overview and security risks. *International Journal of Advanced Research in Science, Communication and Technology, 7*(1), 305-309.

Sharpe, R., Warnicke, E., & Lamping, U. (n.d.). *Wireshark user's guide version 4.1.0*. Wireshark. https://www.wireshark.org/docs/wsug_html_chunked/

Swyx. (2020a, February 23). *Networking essentials: Software defined networking*. DEV. https://dev.to/swyx/networking-essentials-software-defined-networking-1631

Swyx. (2020b, February 23). *Networking essentials: Traffic engineering*. DEV. https://dev.to/swyx/networking-essentials-traffic-engineering-2690

NetworkTutor. (2022, October 1). *Wireshark - Beginners guide - 101 | How to install and capture packets | How to filter ICMP | TCP* [Video]. YouTube. https://www.youtube.com/watch?v=Ud0QK0TPu4U